

Measuring HPC Efficiency with PEAK: How Well Do Your Tools Use Supercomputer Resources?

Caleb Winyninger

*Dept. of Computer Science, Oklahoma City University
REU Intern, Texas Advanced Computing Center
Oklahoma City, OK / Austin, TX, USA
cwiyninger@my.okcu.edu*

Chun-Yaung Lu (Albert)

*Texas Advanced Computing Center
The University of Texas at Austin
Austin, TX, USA
alu@tacc.utexas.edu*

Yinzhi Wang (Ian)

*Texas Advanced Computing Center
The University of Texas at Austin
Austin, TX, USA*

Abstract—High Performance Computing (HPC) systems and the science, math, and machine learning programming libraries optimized for those systems are essential for accelerating large-scale scientific research. Maximizing the performance of these libraries is critical for enabling faster scientific discovery. Using the Performance Evaluation and Analysis Kit (PEAK) and usage data gathered from the Texas Advanced Computing Center (TACC), this project profiles several widely utilized science and math toolkits, including ABINIT, DFTB+, and GROMACS, to evaluate how efficiently they utilize standard numerical libraries such as BLAS, LAPACK, and FFTW across the diverse operational contexts common to HPC environments. By employing a cost-adaptive profiling strategy that balances accuracy and overhead trade-offs via dynamic library preloading, the expected outcome is to capture critical performance data points and library invocation frequencies that traditional profilers miss. At the conclusion of testing, the expectation is to identify specific computational bottlenecks and ideal operational conditions to deliver concrete optimization recommendations. This analysis will provide a definitive framework for the future design of high-performance computing infrastructure and tools prioritizing maximum runtime efficiency, ultimately reducing core-hour expenditure and accelerating computational workflows for the broader scientific community.

Index Terms—high performance computing, profiling, PEAK, BLAS, LAPACK, FFTW, benchmarking, scientific computing

I. INTRODUCTION

A. Broad Context

High Performance Computing (HPC) systems are critical tools for accelerating large scale scientific research. These systems are built from many interconnected computers referred to as nodes. Due to this unique architecture, programs must be designed specifically to take advantage of these resources.

B. The Problem

In order to optimize for these environments, many things must be known about a program: what other software it relies on, where a majority of computation time is spent, how different amounts and types of resources affects performance, and the impact of different variables on performance.

C. The Solution

Using usage data from the Texas Advanced Computing Center (TACC), the Performance Evaluation and Analysis Kit (PEAK), and the Stampede3 supercomputer, we can analyze some of the most used scientific software to determine how well they utilize HPC resources and get a deeper insight into where the programs are spending their time.

D. Roadmap

This paper will provide an overview of how modern HPC systems work, what problems they introduce for optimization, where these programs spend their time, what libraries these programs rely on, and how well these programs take advantage of HPC resources. Section II will provide a brief overview of HPC systems and the challenges they introduce as well as an introduction to the PEAK tool. Section III will provide an overview of the approach of how we select, build, test, scale, and analyze the programs. Section IV will present our results. Section V will provide a discussion of our results and what they mean for the programs we analyzed. Section VI will highlight some of our limitations and explain how we plan to address them among other future work. Section VII will provide a conclusion and summary of our findings.

II. BACKGROUND

A. HPC Systems

Most modern HPC systems are built upon a cluster computing architecture, this means that they are made up of many interconnected computers referred to as nodes. These nodes vary in hardware, purpose, and performance but all are meant to work together to solve large scale problems. In order to run a program on an HPC system, a scheduler is used to allocate resources and manage the execution of the program. A user must submit a job script to the scheduler that specifies the resources needed and the program to be run and the scheduler will then allocate the resources and run the program on the allocated nodes.

B. Prior Work

1) *PEAK*: PEAK is a tool designed for HPC systems that allows the user to analyze a target program for function calls, where compute time is spent, and how the targeted program utilizes popular mathematical libraries such as BLAS (Basic Linear Algebra Subprograms), LAPACK (Linear Algebra PACKage), and FFTW (Fastest Fourier Transform in the West). PEAK uses the Frida-Gum toolkit to inject monitoring code into running binaries and dynamically attaches and detaches to stay below a specified level of overhead. This allows for PEAK to provide more accurate results while still keeping overhead low. PEAK is configured via environmental variables and invoked via `LD_PRELOAD`. Due to PEAK being in active development, this study also provides useful feedback directly to the PEAK team on usage experience, documentation, and bugs.

2) *BenchPRO*: Benchmark Performance and Reproducibility Orchestrator (BenchPRO) is a tool designed to run reproducible benchmarks. The goals of BenchPRO and this study overlap, but we opted for a from-scratch approach in order to have more control over individual configurations and to better understand the underlying processes and techniques for performance benchmarking.

III. APPROACH

A list of the 100 program names, sorted by core hour used, gathered from the Frontera supercomputer was trimmed of unactionable names such as duplicates, language interpreters, or generic names into a list of actionable names. We added more programs to this list based on our own knowledge of popular scientific software. cursory research was done on each program in order to determine domain, purpose, and if TACC had a prebuilt version of the program available. LLMs were used to assist in this research. There are currently 67 programs and counting in this list.

A. Environment

All programs were run on the Stampede3 supercomputer at TACC using nodes equipped with 48 core Intel Xeon Platinum 8160 (“Skylake”) processors and 192 GB of RAM.

B. Build Scripts and Modules

Some programs were available as prebuilt modules maintained by TACC, while others required building from source. In order to keep binaries reproducible and identical, we opted for TACC’s modules when available and created build scripts when they were not.

C. Test Cases and Input

For each program, we chose or created a test case that would run in a reasonable amount of time and represent a typical use case for the program. When available, we sourced our test cases from a program’s pre-bundled test cases or third party test cases. When those sources were unavailable, we created our own test cases. Due to the in-depth knowledge required to create or modify a test case for a program, LLMs were used to assist in creating and modifying test cases.

D. Scaling

Programs were run at a series of different scales, starting with a single node and scaling up to 16 nodes. Single node scaling was done at 1, 24, and 48 cores. Multi-node scaling was done at 2, 8, and 16 nodes with 48 cores per node.

E. Automation Scripts

In order to speed up research, a reusable bash script was created to automatically generate the needed job scripts for each program, scale, and test case. The script read from a configuration file that specified the program, scale, and test cases to be run. Another job was created to run the target program with PEAK attached and configured to profile for BLAS, LAPACK, and FFTW function calls. Another script was created to consolidate similar jobs into job arrays to reduce the number of jobs submitted to the scheduler, allowing for more jobs to be submitted at once. All jobs were organized into directories based on program name, scale, and test case for easy access and analysis. All jobs included timing logging and automatically appended data to a main summary file.

F. Visualization

In order to understand our results, we created a python script to parse the summary files and generate visualizations of the data.

IV. RESULTS

A. ABINIT

Description: Simulates how electrons and atoms behave in materials, using quantum mechanics to predict a material’s structure, energy, and properties.

Test Case: Simulating a silicon crystal.

TABLE I
ABINIT JOB SUMMARY

Config	Nodes	Tasks	PEAK	Elapsed (s)	Job ID
n1_nopeak	1	1	No	119	3294914
n24_nopeak	1	24	No	10	3294915
n48_nopeak	1	48	No	8	3294916
N2_nopeak	2	96	No	6	3294917
N8_nopeak	8	384	No	6	3294918
N16_nopeak	16	768	No	9	3294919
n1_peak	1	1	Yes	126	3294920

TABLE II
ABINIT PEAK RESULTS

Function	Calls	Total Time (s)	Avg. Time /Call (ms)	PEAK Overhead (s)
zgemm_	207,990	4.7613	0.0229	0.4258
ddot_	216,544	0.6383	0.0029	0.4433
zcopy_	131,022	0.6081	0.0046	0.2682
dznrm2_	116,280	0.5958	0.0051	0.2380
zdotc_	51,588	0.1846	0.0036	0.1056

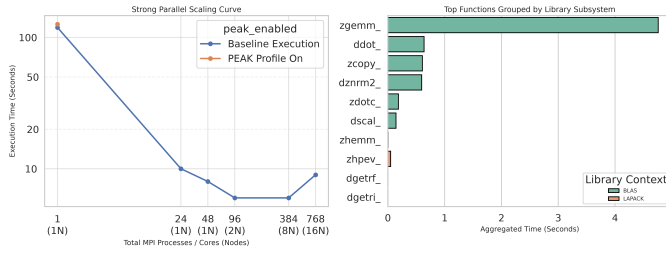


Fig. 1. ABINIT scaling and PEAK results.

B. Quantum Espresso

Description: Quantum-mechanical materials simulation package, solving the same kind of problem as ABINIT with a different underlying computational method.

Test Case: Simulating a silicon crystal.

TABLE III
QUANTUM ESPRESSO JOB SUMMARY

Config	Nodes	Tasks	PEAK	Elapsed (s)	Job ID
n1_nopeak	1	1	No	636	3306688
n24_nopeak	1	24	No	63	3306689
n48_nopeak	1	48	No	34	3306690
N2_nopeak	2	96	No	28	3306691
N8_nopeak	8	384	No	20	3306692
N16_nopeak	16	768	No	27	3306693
n1_peak	1	1	Yes	638	3306694

TABLE IV
QUANTUM ESPRESSO PEAK RESULTS

Function	Calls	Total Time (s)	Avg. Time /Call (ms)	PEAK Overhead (s)
zgemm_	144,849	166.6082	1.1502	0.2227
zhgvr_	5,561	8.3896	1.5087	0.0085
zgemv_	1,184	3.7863	3.1979	0.0018
ddot_	57,984	0.3923	0.0068	0.0891
dcopy_	1,926	0.3207	0.1665	0.0030

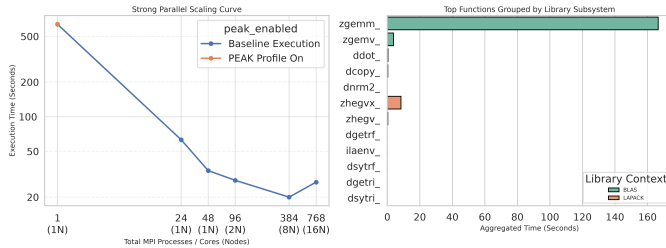


Fig. 2. Quantum Espresso scaling and PEAK results.

C. DFTB+

Description: A faster, approximate alternative to full quantum-mechanical simulation, trading some accuracy for greatly reduced compute time.

Test Case: Simulating a carbon molecule.

TABLE V
DFTB+ JOB SUMMARY

Config	Nodes	Tasks	PEAK	Elapsed (s)	Job ID
n1_nopeak	1	1	No	120	3303873
n24_nopeak	1	24	No	144	3303874
n48_nopeak	1	48	No	220	3303875
n1_peak	1	1	Yes	121	3303879
N2_nopeak	2	96	No	2*	3303876
N8_nopeak	8	384	No	3*	3303877
N16_nopeak	16	768	No	3*	3303878

* Job exited with a non-zero exit code (1); reported elapsed time reflects time-to-crash, not completion.

TABLE VI
DFTB+ PEAK RESULTS

Function	Calls	Total Time (s)	Avg. Time /Call (s)	PEAK Overhead (s)
dsyev_	4	94.1030	23.5258	0.0000
dgemm_	442	10.5098	0.0238	0.0007
dsymm_	4	10.0370	2.5092	0.0000
dlarrv_	4	0.1037	0.0259	0.0000
dcopy_	34,760	0.0280	0.0008	0.0541

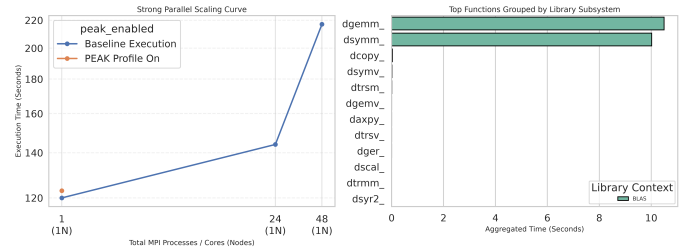


Fig. 3. DFTB+ scaling and PEAK results.

D. GROMACS

Description: Simulates the motion and interactions of biomolecules like proteins and lipids, widely used to study biological and chemical processes.

Test Case: Simulating protein in a membrane.

TABLE VII
GROMACS JOB SUMMARY

Config	Nodes	Tasks	PEAK	Elapsed (s)	Job ID
n24_nopeak	1	24	No	213	3295859
n48_nopeak	1	48	No	125	3295860
n48_peak	1	48	Yes	126	3295864
N2_nopeak	2	96	No	75	3295861
N8_nopeak	8	384	No	37	3295862
N16_nopeak	16	768	No	36	3295863

PEAK Results: This build of GROMACS did not yield any PEAK hits.

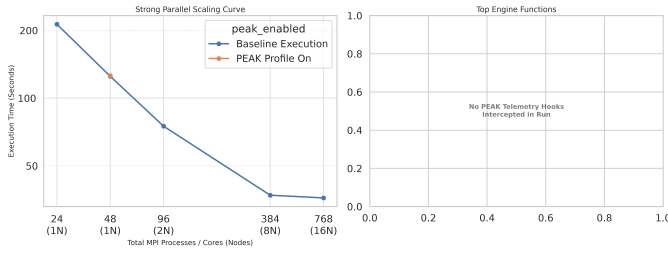


Fig. 4. GROMACS scaling results.

E. LAMMPS

Description: Simulates how large numbers of atoms and molecules move and interact over time (molecular dynamics), commonly used to study materials at the atomic scale.

Test Case: Simulating protein in a membrane.

TABLE VIII
LAMMPS JOB SUMMARY

Config	Nodes	Tasks	PEAK	Elapsed (s)	Job ID
n1_nopeak	1	1	No	32	3305579
n24_nopeak	1	24	No	4	3305580
n48_nopeak	1	48	No	4	3305581
N2_nopeak	2	96	No	4	3305582
N8_nopeak	8	384	No	4	3305583
N16_nopeak	16	768	No	8*	3305584
n1_peak	1	1	Yes	32	3305585

*Job exited with a non-zero exit code (255).

PEAK Results: This build of LAMMPS did not yield any PEAK hits.

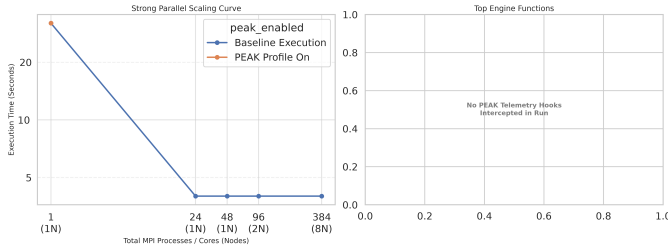


Fig. 5. LAMMPS scaling results.

V. DISCUSSION

A. ABINIT

ABINIT scaled effectively from 1 to 384 tasks (119s to 6s), but got slightly worse at 768 tasks (9s). This is most likely due to inter-node communication overhead. This is a common problem in HPC systems, where the time spent communicating between nodes can outweigh the benefits of adding more resources. This run represents what we would expect from most programs, where more resources leads to better performance, but only up to a certain point. ABINIT relies heavily on BLAS and somewhat on LAPACK. A majority of compute is spent on the `zgemm_` function, which is a BLAS function for matrix multiplication.

B. Quantum Espresso

Quantum Espresso scales well up to 384 tasks (636s to 20s), then degrades at 768 tasks (27s). This is a similar story to ABINIT, where the program scales well up to a certain point, but then suffers from inter-node communication overhead. Quantum Espresso's PEAK results resemble ABINIT's, with a majority of compute time spent on the `zgemm_` function, which is a BLAS function for matrix multiplication.

C. DFTB+

The most interesting result from our study was DFTB+, which performed worse with more resources and outright crashed when multiple nodes were introduced. This is a clear example of a program that does not scale well and is not designed to take advantage of HPC resources. These results tell us that DFTB+ or its test case is not designed to take advantage of multiple nodes and is not suitable for running on an HPC system. DFTB+ relies on BLAS, with a majority of compute time spent on `dsymm_` and `zgemm_`, `dsymm_` being another BLAS function for matrix multiplication.

D. GROMACS

GROMACS scaled the best out of the group, successfully taking advantage of all the resources we gave it. GROMACS did not yield PEAK hits, which is likely due to how it was compiled. Rebuilding GROMACS under different configurations may yield PEAK results. GROMACS is an example of what we would expect from a well optimized HPC program, where more resources leads to better performance.

E. LAMMPS

LAMMPS plateaus fast (32s to 4s by 24 tasks) and stays flat through 384 tasks. The program or test case seems to impose a hard limit on what resources the program allows itself to attempt to use. This is an example of a program or test case that avoids situations where inter-node communication overhead can become a problem by not allowing itself to use more resources than it can effectively utilize. LAMMPS did not yield PEAK hits, which is likely due to how it was compiled. Rebuilding LAMMPS under different configurations may yield PEAK results.

F. General Discussion

To cover a broad survey of Frontera's most-used programs, a breadth-first approach was taken over deep per-program investigation, leaving room to resolve profiling issues and test more permutations for deeper insight. For the programs that yielded no PEAK results, it is likely due to how they were compiled. Rebuilding the programs under different configurations may yield PEAK results.

VI. LIMITATIONS AND FUTURE WORK

The main limitation of this study is the breadth-first approach taken, which left room for deeper investigation into each program. Future work will include rebuilding programs under different configurations to yield PEAK results, testing

more permutations of test cases and scales, and investigating the programs that did not scale well to determine if it is a problem with the program itself or the test case used. It is planned to expand this study from an REU (Research Experience for Undergraduates) project to a full research project, with the goal of publishing the results in a peer-reviewed journal. With more time, and using the knowledge gained from this study, we can investigate more programs and test more permutations to get a deeper insight into how well these programs utilize HPC resources. BenchPRO is also a path that can be explored and could improve the research workflow.

VII. CONCLUSION

This study showcases the need for deeper understanding of how software runs on HPC systems and the need for a deeper understanding. Many researchers intuitively expect that more resources means a faster turnaround from their programs, from our results we can see this is not always true.

ACKNOWLEDGMENT

This research and experience would not have been possible without the funding, resources, and support provided by:

- Texas Advanced Computing Center (TACC)
- National Science Foundation (NSF)
- NSF REU Site: Cyberinfrastructure Research for Societal Advancement REU, Award # 2447887
- PEAK Project Award ID: OAC-2402542
- Frontera Supercomputer
- Stampede3 Supercomputer
- University of Texas at Austin (UT)
- Oklahoma City University (OCU)
- Dr. Rosalia Gomez — Education and Outreach Directorate at TACC and REU Program Coordinator
- Dr. Chun-Yaung Lu (Albert) — Research Associate at TACC and Mentor
- Dr. Yinzhi Wang (Ian) — Research Associate at TACC and Mentor
- Bobby Reed — Professor at OCU
- Dr. Chenguang Xu (Shine) — Professor at OCU

REFERENCES

- [1] PEAK team, “PEAK: Lightweight, versatile performance evaluation tool,” 2024. [Online]. Available: <https://github.com/peak-team/peak>
- [2] Y. Chen, J. Li, C.-Y. Lu, and Y. Wang, “PEAK: Cost-Adaptive Profiling in a Heartbeat,” in *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops 25)*, 2025. doi: 10.1145/3731599.3767521
- [3] Y. Wang and J. Li, “PEAK: a Light-Weight Profiler for HPC Systems,” in *Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W 2023)*, 2023. doi: 10.1145/3624062.3624143
- [4] D. Stanzione, J. West, R. T. Evans, T. Minyard, O. Ghattas, and D. K. Panda, “Frontera: The Evolution of Leadership Computing at the National Science Foundation,” in *Practice and Experience in Advanced Research Computing 2020: Catch the Wave*, 2020. doi: 10.1145/3311790.3396656
- [5] Texas Advanced Computing Center (TACC), “Stampede3 User Guide,” 2024. [Online]. Available: <https://docs.tacc.utexas.edu/hpc/stampede3/>
- [6] A. H. Romero et al., “ABINIT: Overview and focus on selected capabilities,” *The Journal of Chemical Physics*, vol. 152, no. 12, p. 124102, 2020. doi: 10.1063/1.5144261
- [7] P. Giannozzi et al., “QUANTUM ESPRESSO: a modular and open-source software project for quantum simulations of materials,” *Journal of Physics: Condensed Matter*, vol. 21, no. 39, p. 395502, 2009. doi: 10.1088/0953-8984/21/39/395502
- [8] M. J. Abraham et al., “GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers,” *SoftwareX*, vol. 1–2, pp. 19–25, 2015. doi: 10.1016/j.softx.2015.06.001
- [9] A. P. Thompson et al., “LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales,” *Computer Physics Communications*, vol. 271, p. 108171, 2022. doi: 10.1016/j.cpc.2021.108171
- [10] B. Hourahine et al., “DFTB+, a software package for efficient approximate density functional theory based atomistic simulations,” *The Journal of Chemical Physics*, vol. 152, no. 12, p. 124101, 2020. doi: 10.1063/1.5143190
- [11] M. Cawood and S. L. Harrell, “BenchPRO: Benchmark Performance & Reproducibility Orchestrator,” Texas Advanced Computing Center (TACC), GitHub repository, 2021–2026. [Online]. Available: <https://github.com/TACC/benchpro>